

VEHICLE RENTAL SYSTEM

Project Report

Student: Ankit Nag

Program: B.Tech CSE (AI & ML)

Institution: VIT Bhopal

1. Abstract

The Vehicle Rental System is a Java console application developed to manage the basic operations of a vehicle rental service. It manages vehicles, customers, rentals, returns, billing, and rental history. The application demonstrates core Java programming and Object-Oriented Programming concepts while using collections and file handling for data management.

2. Introduction

Managing rental records manually can make searching, updating, and maintaining rental information difficult. This project provides a simple console-based system in which vehicle and customer information can be stored and rental operations can be performed through a menu-driven interface.

3. Objectives

- Manage vehicle information efficiently.
- Maintain customer records.
- Rent and return vehicles.
- Calculate rental bills based on daily rent and rental duration.
- Maintain rental history.
- Store and retrieve data using files.
- Demonstrate Java OOP and collection concepts.

4. Technologies Used

Programming Language: Java

IDE: IntelliJ IDEA

Collections: ArrayList

File Handling: FileWriter, FileReader, BufferedReader

Exception Handling: IOException

5. System Features

Vehicle Management: Add, display, search, and remove vehicles.

Vehicle Types: Vehicle, Car, and Bike classes are supported.

Customer Management: Add, display, and search customer records.

Rental Management: Rent available vehicles and calculate the total bill.

Return Management: Mark a rented vehicle as available again.

Rental History: Maintain previously created rental records.

Persistence: Save and load vehicles, customers, and rentals using text files.

6. Class Description

Vehicle: Stores vehicle ID, brand, model, rent per day, and availability.

Car: Inherits Vehicle and adds number of seats.

Bike: Inherits Vehicle and adds engine capacity in CC.

Customer: Stores customer ID, name, and phone number.

Rental: Stores rental ID, customer reference, vehicle reference, and number of days. It calculates the bill.

Main: Controls application execution, collections, menu operations, searching, rental operations, and file handling.

7. Object-Oriented Programming Concepts

Classes and Objects: The system models vehicles, customers, and rentals as Java classes and objects.

Inheritance: Car and Bike extend the Vehicle class.

Method Overriding: Car and Bike override displayVehicle().

Polymorphism: Vehicle references can refer to Car and Bike objects, allowing the overridden method to execute according to the actual object.

Composition: A Rental object contains references to a Customer and a Vehicle.

8. Collections

The project uses ArrayList to dynamically store vehicles, customers, and rental records. This allows records to be added and removed during program execution and supports iteration for searching and displaying data.

9. File Handling

The application uses three text files: vehicles.txt, customers.txt, and rentals.txt. FileWriter is used to save records, while FileReader and BufferedReader are used to load records. Rental records store customer and vehicle IDs and are reconstructed using the corresponding objects in the collections.

10. Rental Bill Calculation

The total rental bill is calculated using:

Total Bill = Rent Per Day × Number of Days

For example, if a vehicle costs ■2,500 per day and is rented for 3 days, the calculated bill is ■7,500.

11. Input Validation

The system checks important inputs such as positive rent, positive rental duration, positive number of seats, positive engine capacity, and duplicate vehicle/customer IDs. It also prevents removal of a vehicle that is currently rented.

12. Application Flow

1. Start the application.
2. Create the required ArrayLists.
3. Load previously saved data.
4. Display the menu.
5. Perform the operation selected by the user.
6. Save data when requested or during the final exit flow.
7. Close the application.

13. Sample Menu

1. Display Vehicles
2. Add Vehicle
3. Search Vehicle
4. Remove Vehicle
5. Add Customer

6. Search Customer
7. Rent Vehicle
8. Return Vehicle
9. Rental History
10. Save Data
11. Exit

14. Advantages

- Simple and easy-to-use console interface.
- Demonstrates important Java programming concepts.
- Dynamic record management using ArrayList.
- Persistent storage using text files.
- Separate classes improve code organization and maintainability.

15. Limitations and Future Scope

The current version is a console-based academic project and uses text files instead of a database. Future versions could include a database, graphical or web interface, login/authentication, dates and timestamps, advanced search/filtering, payment integration, and richer vehicle categories.

16. Conclusion

The Vehicle Rental System successfully demonstrates how Java OOP, collections, methods, file handling, and basic exception handling can be combined to create a functional rental management application. The project provides the core operations required to manage vehicles, customers, and rental records in a simple and organized manner.